



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №8

по дисциплине

«Разработка баз данных»

Выполнил студент группы ИНБО-20-23

Школьный И.В.

Проверил доцент

Исаева М.В.

Москва 2025

ПРАКТИЧЕСКАЯ РАБОТА №8. СЛОЖНЫЕ ТИПЫ ДАННЫХ И МАСШТАБИРОВАНИЕ POSTGRESQL

Цель работы:

Целью данной практической работы является освоение методов работы со сложными и неструктурированными типами данных (JSONB, BYTEA) в PostgreSQL, а также получение практических навыков реализации горизонтального масштабирования базы данных с помощью декларативного секционирования.

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить следующие шаги, адаптируя примеры из БД «Аптека» к вашей собственной базе данных.

Задание №1: хранение паролей (BYTEA)

1. Создать справочную таблицу ролей и таблицу системных пользователей (или модифицировать существующую).
2. В таблице пользователей предусмотреть поля для хранения «сырого» пароля (только для учебной демонстрации) и его хеша (тип BYTEA).
3. Сохранить хеши паролей пользователей, используя функцию хеширования `digest`.
4. Выполнить запрос, проверяющий соответствие введенного пароля сохраненному хешу. Одна проверка должна быть успешной, другая - нет (допустимо сделать это в одном запросе).

Задание №2: секционирование таблицы логов

1. Создать новую секционированную таблицу для хранения логов событий.
2. Использовать стратегию секционирования по диапазонам (RANGE) по полю даты.
3. Создать две секции для периодов:
 - 2-е полугодие 2024 (с 2024-07-01 по 2025-01-01).
 - 1-е полугодие 2025 (с 2025-01-01 по 2025-07-01).

4. Создать секцию по умолчанию, куда будут попадать все данные, выходящие за рамки указанных диапазонов (будущее время).

Задание №3: генерация данных

Написать скрипт на PL/pgSQL для генерации 20 000 записей в диапазоне дат, покрывающем созданные секции. Сгенерированные данные должны содержать JSON-структуру с различным набором полей для разных типов событий (например, чтобы логи «продажи» отличались по структуре от логов «входа»).

Вывести сгенерированные данные – показать запросом SELECT, что данные успешно созданы и физически распределены по разным таблицам-секциям.

Задание №4: анализ и поиск по JSONB

Написать три запроса:

1. Фильтрация по значению – найти записи, где числовое поле внутри JSON удовлетворяет математическому условию (например, > 800), используя операторы извлечения $\rightarrow$ и $\rightarrow>$.

2. Поиск по вхождению – найти записи, содержащие точный фрагмент JSON-структуры (например, `{"quantity": 5}`), используя оператор `@>`.

3. Агрегация – посчитать сумму или среднее значение числового поля из JSON-документа.

Задание №5: модификация данных JSONB

Продемонстрировать изменение данных внутри JSON-поля:

1. Массовое добавление нового поля во все записи определенного типа.
2. Точечное изменение значения по ключу в конкретной записи.

ХОД ВЫПОЛНЕНИЯ РАБОТЫ

В данной работе мы расширяем функционал базы данных, добавляя подсистему безопасности и аудита. Мы будем использовать таблицы из предыдущих работ для контекста, но создадим новые таблицы для пользователей системы и журнала событий.

Ниже приводятся таблицы, используемые для построения запросов.

Таблица 1. Таблица Бронирование (Booking)

Auto

postgres shkolnyi_iv@dbstud

<postgres> pr7 booking

СвойстваДанныеДиаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	123 room_id	check_in_date	check_out_date	123 loyalty_id
1	1	1	1	2023-06-01	2023-06-10	1
2	2	2	2	2023-07-15	2023-07-20	2
3	3	3	3	2023-08-05	2023-08-15	3
4	4	4	4	2023-09-01	2023-09-05	1
5	5	5	5	2023-09-10	2023-09-15	2
6	6	6	6	2023-09-20	2023-09-25	3
7	7	7	7	2023-10-01	2023-10-07	1
8	8	8	8	2023-10-15	2023-10-20	2
9	9	9	9	2023-11-05	2023-11-12	3
10	10	10	10	2023-11-20	2023-11-25	1
11	11	1	1	2024-01-15	2024-01-20	1
12	12	1	1	2024-02-01	2024-02-04	1

Таблица 2. Таблица Документы о бронировании (Booking_document)

▼Auto

↺

🔒

🖨️postgres

▼

📄shkolnyi_iv@dbstud

▼

🕒

▼

📄

▼

🔍

▼

🖨️<postgres> pr7

📄booking_document

✕

📄Свойства

📄Данные

📄Диаграмма

📄Показать SQL

🔍Введите SQL выражение чтобы отфильтровать результаты

Таблица

Текст

	123 id	123 booking_id	🕒 issue_date	A-Z status
1	1	1	2023-05-15	Confirmed
2	2	2	2023-06-20	Paid
3	3	3	2023-07-10	Completed
4	4	4	2023-08-15	Confirmed
5	5	5	2023-08-25	Paid
6	6	6	2023-09-10	Completed
7	7	7	2023-09-20	Confirmed
8	8	8	2023-10-05	Paid
9	9	9	2023-10-25	Completed
10	10	10	2023-11-10	Confirmed
11	11	11	2025-10-27	Confirmed
12	12	12	2024-01-20	Confirmed

Таблица 3. Таблица Брошюра (Brochure)

uto

postgres

shkolnyi_iv@dbstud

brochure

Свойства

Данные

Диаграмма

Показать SQL

Введите SQL выражение чтобы отфильтровать результаты

Таблица

Текст

	123 id	123 hotel_id	A-Z language	publication_date
1	1	1	Russian	2022-01-10
2	2	2	English	2022-03-15
3	3	3	German	2022-05-20
4	4	1	English	2022-02-28
5	5	2	German	2022-04-10
6	6	3	Russian	2022-06-15
7	7	1	German	2022-07-20
8	8	2	English	2022-08-25
9	9	3	English	2022-09-30
10	10	1	Russian	2022-10-05
11	11	2	Russian	2022-11-12
12	12	3	German	2022-12-18

Таблица 4. Таблица Регистрация (Check_in)

Auto postgres shkolnyi_iv@dbstud

<postgres> pr7 check_in X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	123 guest_id	123 total_bill	123 room_id	123 doc_id
1	1	1	45 000	1	1
2	2	2	42 500	2	2
3	3	3	120 000	3	3
4	4	4	18 000	4	4
5	5	5	39 000	5	5
6	6	6	57 500	6	6
7	7	7	31 200	7	7
8	8	8	41 000	8	8
9	9	9	84 000	9	9
10	10	10	27 500	10	10
11	11	1	12 300	1	12

Таблица 5. Таблица Работник (Employee)

uto postgres shkolnyi_iv@dbstud

employee X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z full_name	123 position_id	hire_date	123 hotel_id
1	1	Смирнова Ольга Дмитриевна	1	2018-06-10	1
2	2	Кузнецов Андрей Петрович	2	2020-02-15	2
3	3	Васильева Елена Ивановна	3	2021-05-20	3
4	4	Петров Максим Игоревич	4	2019-03-12	1
5	5	Семенова Ирина Васильевна	5	2020-08-25	2
6	6	Ковалев Александр Петрович	6	2018-11-30	3
7	7	Новикова Юлия Сергеевна	7	2021-02-14	1
8	8	Данилов Алексей Викторович	8	2017-07-22	2
9	9	Тихонова Марина Олеговна	9	2016-05-18	3
10	10	Фролов Денис Николаевич	10	2022-01-10	1

Таблица 6. Таблица Гость (Guest)

Auto postgres shkolnyi_iv@dbstud

guest X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z full_name	A-Z phone_number	A-Z email
1	1	Козлова Мария Игоревна	7911123456	kozlova@mail.ru
2	2	Николаев Дмитрий Олегович	7902876543	nikolaev@gmail.com
3	3	Федорова Екатерина Павловна	7922555777	fedorova@yandex.ru
4	4	Громов Сергей Александрович	7913987654	gromov@mail.ru
5	5	Волкова Анастасия Викторовна	7904765432	volkova@gmail.com
6	6	Белов Артем Сергеевич	7923876541	belov@yandex.ru
7	7	Соколова Ольга Николаевна	7915234567	sokolova@mail.ru
8	8	Иванов Иван Иванович	7910123456	ivanov@mail.ru
9	9	Петрова Анна Сергеевна	7909876543	petrova@gmail.com
10	10	Сидоров Алексей Викторович	7921555777	sidorov@yandex.ru

Таблица 7. Таблица Документы гостя (Guest_document)

uto postgres shkolnyi_iv@dbstud

guest_document X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	123 guest_id	A-Z document_data	A-Z document_type	issue_date	expiry_date
1	1	1	1234567890	Passport	2015-05-20	2025-05-20
2	2	2	0987654321	Birth certificate	2020-01-15	2030-01-15
3	3	3	1122334455	Driver license	2018-11-30	2028-11-30
4	4	4	4455667788	Passport	2017-03-10	2027-03-10
5	5	5	5566778899	Driver license	2019-08-22	2029-08-22
6	6	6	6677889900	Passport	2016-11-05	2026-11-05
7	7	7	7788990011	Birth certificate	2021-02-14	2031-02-14
8	8	8	8899001122	Driver license	2018-07-30	2028-07-30
9	9	9	9900112233	Passport	2019-12-20	2029-12-20
10	10	10	1011223344	Birth certificate	2020-04-18	2030-04-18

Таблица 8. Таблица Гостиница (Hotel)

uto postgres shkolnyi_iv@dbstud

guest_document hotel X

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

Таблица	123 id	A-Z name	A-Z address	123 star_rating	A-Z description
1	1	City Lights	ул. Невский проспект, 45	4	Современный отель в историческом центре
2	2	River Side	наб. реки Мойки, 12	5	Элитный отель с видом на реку
3	3	Park Inn	ул. Садовая, 78	3	Комфортабельный отель рядом с парком

Таблица 9. Таблица Программа лояльности (Loyalty_program)

auto postgres shkolnyi_iv@dbstud

loyalty_program

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z name	123 discount	A-Z description
1	Gold Member	15	Exclusive discounts and early check-in for gold members
2	Silver Member	10	Special offers and room upgrades for silver members
3	Bronze Member	5	Basic discounts for frequent guests

Таблица 10. Таблица Должность (Position)

auto postgres shkolnyi_iv@dbstud

loyalty_program position

Свойства Данные Диаграмма

Показать SQL Введите SQL выражение чтобы отфильтровать результаты

123 id	A-Z title	123 base_rate
1	Менеджер	50 000
2	Портье	35 000
3	Горничная	30 000
4	Администратор	45 000
5	Шеф-повар	80 000
6	Официант	32 000
7	Охранник	38 000
8	Уборщик	28 000
9	Бухгалтер	55 000
10	Маркетолог	48 000
11	IT-специалист	60 000
12	Директор	100 000
13	Аниматор	35 000

Таблица 11. Таблица Номер (Room)

auto

postgres

shkolnyi_iv@dbstud

loyalty_program

position

room X

Свойства

Данные

Диаграмма

Показать SQL

Введите SQL выражение чтобы отфильтровать результаты

	123 id	A-Z room_number	A-Z type	123 price_per_night	123 hotel_id
1	1	101	Standard	5 000	1
2	2	202	Deluxe	8 500	2
3	3	301	Suite	12 000	3
4	4	102	Standard	4 500	1
5	5	103	Deluxe	7 500	1
6	6	201	Standard	4 800	2
7	7	202	Deluxe	7 800	2
8	8	203	Suite	11 500	2
9	9	301	Standard	5 200	3
10	10	302	Deluxe	8 200	3
11	11	303	Suite	12 500	3
12	12	401	Standard	5 500	1

Задание 1. Хранение паролей (BYTEA)

В БД уже есть таблицы с персоналом и должностями. Необходимо внести небольшие корректировки в таблицу персонала, добавив строки с паролем.

Листинг 1. Редактирование таблицы employee

```
alter table employee add column if not exists password_raw TEXT;
alter table employee add column if not exists password_hash BYTEA;

-- Обновляем пароли для существующих сотрудников (учебные пароли)
UPDATE Employee SET
  password_raw = CASE id
    WHEN 1 THEN 'ManagerPass123!' -- Смирнова Ольга Дмитриевна (менеджер)
    WHEN 2 THEN 'PorterPass456!' -- Кузнецов Андрей Петрович (портье)
    WHEN 3 THEN 'MaidPass789!' -- Васильева Елена Ивановна (горничная)
    WHEN 4 THEN 'AdminPass2024!' -- Петров Максим Игоревич (администратор)
    WHEN 5 THEN 'ChefPass!2024' -- Семенова Ирина Васильевна (шеф-повар)
    WHEN 6 THEN 'AccountantPass' -- Ковалев Александр Петрович (бухгалтер)
    WHEN 7 THEN 'Marketer2024!' -- Новикова Юлия Сергеевна (маркетолог)
    WHEN 8 THEN 'ITspecialist!' -- Данилов Алексей Викторович (IT-специалист)
    WHEN 9 THEN 'DirectorPass' -- Тихонова Марина Олеговна (директор)
    WHEN 10 THEN 'AnimatorFun!' -- Фролов Денис Николаевич (аниматор)
    ELSE 'DefaultPass123'
  END,
  password_hash = CASE id
    WHEN 1 THEN digest('ManagerPass123!', 'sha256')
    WHEN 2 THEN digest('PorterPass456!', 'sha256')
    WHEN 3 THEN digest('MaidPass789!', 'sha256')
    WHEN 4 THEN digest('AdminPass2024!', 'sha256')
    WHEN 5 THEN digest('ChefPass!2024', 'sha256')
    WHEN 6 THEN digest('AccountantPass', 'sha256')
    WHEN 7 THEN digest('Marketer2024!', 'sha256')
    WHEN 8 THEN digest('ITspecialist!', 'sha256')
    WHEN 9 THEN digest('DirectorPass', 'sha256')
    WHEN 10 THEN digest('AnimatorFun!', 'sha256')
    ELSE digest('DefaultPass123', 'sha256')
  END
WHERE password_hash IS NULL;
```

Запустим скрипт. Далее выполним проверку успешного хеширования паролей. Теперь напомним запрос, в котором проверим корректность хеширования.

Листинг 2. Проверка хеширования

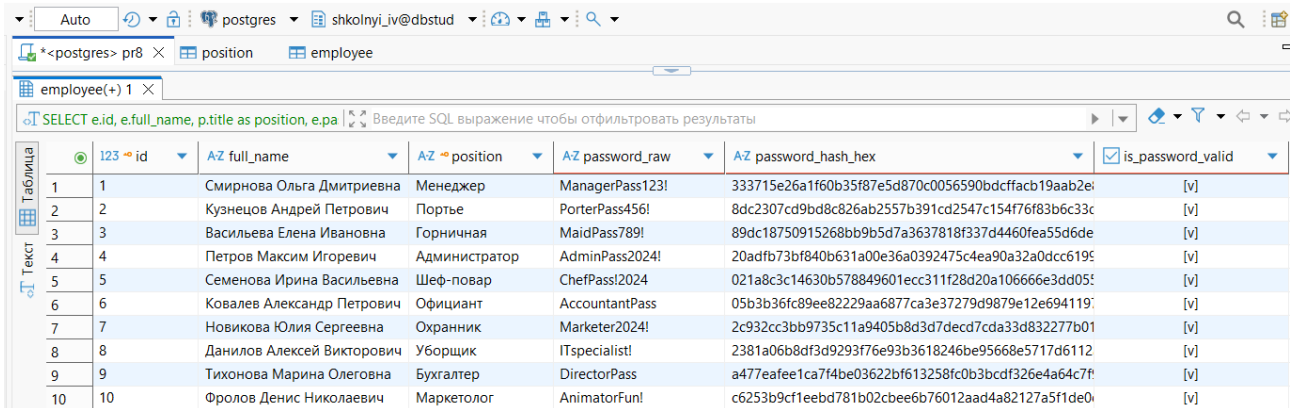
```
SELECT
  e.id,
  e.full_name,
  p.title as position,
  e.password_raw,
```

```

encode(e.password_hash, 'hex') as password_hash_hex,
-- Проверяем соответствие пароля и хеша
(digest(e.password_raw, 'sha256') = e.password_hash) as is_password_valid
FROM Employee e
JOIN Position p ON e.position_id = p.id;

```

Запустим скрипт.



	id	full_name	position	password_raw	password_hash_hex	is_password_valid
1	1	Смирнова Ольга Дмитриевна	Менеджер	ManagerPass123!	333715e26a1f60b35f87e5d870c0056590bdcffac19aab2e	[v]
2	2	Кузнецов Андрей Петрович	Портье	PorterPass456!	8dc2307cd9bd8c826ab2557b391cd2547c154f76f83b6c33c	[v]
3	3	Васильева Елена Ивановна	Горничная	MaidPass789!	89dc18750915268bb9b5d7a3637818f337d4460fea55d6de	[v]
4	4	Петров Максим Игоревич	Администратор	AdminPass2024!	20adfb73bf840b631a00e36a0392475c4ea90a32a0dcc619f	[v]
5	5	Семенова Ирина Васильевна	Шеф-повар	ChefPass!2024	021a8c3c14630b578849601ecc311f28d20a106666e3dd05!	[v]
6	6	Ковалев Александр Петрович	Официант	AccountantPass	05b3b36fc89ee8229aa6877ca3e37279d9879e12e694119!	[v]
7	7	Новикова Юлия Сергеевна	Охранник	Marketer2024!	2c932cc3bb9735c11a9405b8d3d7decdd7cda33d832277b01	[v]
8	8	Данилов Алексей Викторович	Уборщик	ITspecialist!	2381a06b8df3d9293f76e93b3618246be95668e5717d6112	[v]
9	9	Тихонова Марина Олеговна	Бухгалтер	DirectorPass	a477eafec1ca7f4be03622bf613258fc0b3bcd3f326e4a64c7f!	[v]
10	10	Фролов Денис Николаевич	Маркетолог	AnimatorFun!	c6253b9cf1eebd781b02cbee6b76012aad4a82127a5f1de0!	[v]

Рисунок 1 — Проверка хеширования

Задание 2. Секционирование таблицы логов

Создадим таблицу `hotel_event_logs` для хранения истории действий. Используем стратегию разделения данных `RANGE` (по диапазону) для поля `created_at`.

Листинг 3. Таблица логов

```

CREATE TABLE hotel_event_logs (
  log_id BIGSERIAL,
  created_at TIMESTAMPTZ NOT NULL,
  user_id BIGINT REFERENCES Employee(id),
  event_data JSONB
) PARTITION BY RANGE (created_at);

```

Создадим секции согласно заданию: 2-е полугодие 2024 и 1-е полугодие 2025. Также добавим секцию `DEFAULT`, которая будет автоматически принимать все данные, не попадающие в явные диапазоны (например, текущие данные конца 2025 года).

Листинг 4. Создание секций

```

-- Секция 1: 2-е полугодие 2024 (июль-декабрь)
CREATE TABLE hotel_logs_2024_h2 PARTITION OF hotel_event_logs
FOR VALUES FROM ('2024-07-01') TO ('2025-01-01');

-- Секция 2: 1-е полугодие 2025 (январь-июнь)

```

```
CREATE TABLE hotel_logs_2025_h1 PARTITION OF hotel_event_logs  
FOR VALUES FROM ('2025-01-01') TO ('2025-07-01');
```

-- Секция по умолчанию (для всего остального)

```
CREATE TABLE hotel_logs_default PARTITION OF hotel_event_logs DEFAULT;
```

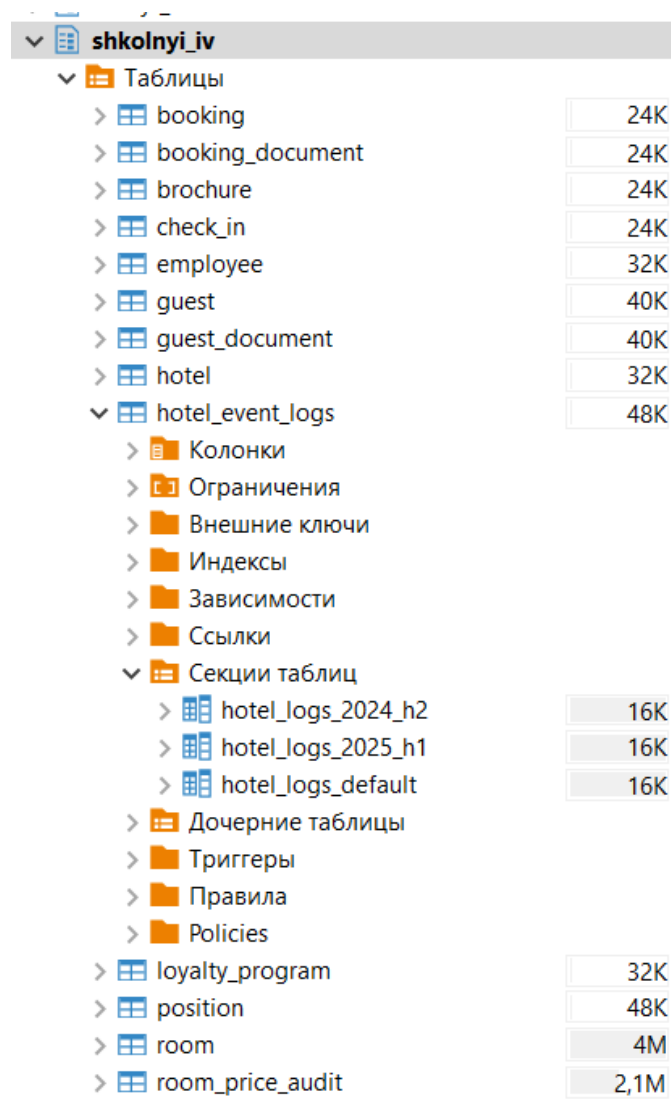
Индексы создаются на родительской таблице и наследуются всеми секциями.

Листинг 4. Создание индекса

-- Индекс автоматически создается для всех секций

```
CREATE INDEX idx_hotel_logs_date ON hotel_event_logs(created_at);
```

Убедимся в корректности выполнения.



shkolnyi_iv	
Таблицы	
booking	24K
booking_document	24K
brochure	24K
check_in	24K
employee	32K
guest	40K
guest_document	40K
hotel	32K
hotel_event_logs	48K
Колонки	
Ограничения	
Внешние ключи	
Индексы	
Зависимости	
Ссылки	
Секции таблиц	
hotel_logs_2024_h2	16K
hotel_logs_2025_h1	16K
hotel_logs_default	16K
Дочерние таблицы	
Триггеры	
Правила	
Policies	
loyalty_program	32K
position	48K
room	4M
room_price_audit	2,1M

Рисунок 2 — Корректное выполнение

Задание 3. Генерация данных и тип JSONB

JSONB — это эффективный формат хранения JSON в PostgreSQL. Он позволяет хранить данные, структура которых может меняться от записи к записи.

Сгенерируем 20 000 записей. Пояснение к генерации данных: поскольку JSON — это гибкая структура, мы можем хранить в нем ссылки на любые сущности нашей базы данных.

Листинг 5. Генерация 20000 записей

```
DO $$
DECLARE
  i INT;
  random_date TIMESTAMPTZ;
  random_user INT;
  event_type TEXT;
  json_payload JSONB;
BEGIN
  RAISE NOTICE 'Начинаем генерацию 20000 записей...';

  FOR i IN 1..20000 LOOP
    -- Случайная дата между июлем 2024 и июнем 2025
    random_date := '2024-07-01'::TIMESTAMPTZ + (random() * 365)::INT * INTERVAL '1
day';

    -- Случайный сотрудник (ID 1-10)
    random_user := floor(random() * 10 + 1)::INT;

    -- Случайный тип события: login или booking (как в методичке)
    IF (i % 2 = 0) THEN
      event_type := 'login';
      -- JSON для входа в систему
      json_payload := jsonb_build_object(
        'event', event_type,
        'ip', '192.168.1.' || floor(random() * 255)::TEXT,
        'success', (random() > 0.1)
      );
    ELSE
      event_type := 'booking';
      -- JSON для бронирования
      json_payload := jsonb_build_object(
        'event', event_type,
        'details', jsonb_build_object(
          'room_id', floor(random() * 10 + 1),
          'nights', floor(random() * 14 + 1),
          'total_sum', (random() * 100000)::NUMERIC(10,2)
        )
      );
    END IF;
```

```

-- Вставляем запись в лог
INSERT INTO hotel_event_logs (created_at, user_id, event_data)
VALUES (random_date, random_user, json_payload);

-- Сообщение о прогрессе
IF i % 4000 = 0 THEN
    RAISE NOTICE 'Добавлено % записей', i;
END IF;
END LOOP;

RAISE NOTICE 'Генерация завершена!';
END;
$$ LANGUAGE plpgsql;

```

Убедимся, что данные «разлетелись» по нужным таблицам.

Листинг 6. Проверка

```

-- Проверяем распределение по секциям (как в методичке)
SELECT '2024_h2' as partition_name, count(*) FROM hotel_logs_2024_h2
UNION ALL
SELECT '2025_h1', count(*) FROM hotel_logs_2025_h1
UNION ALL
SELECT 'default', count(*) FROM hotel_logs_default;

```

Запустим скрипт.

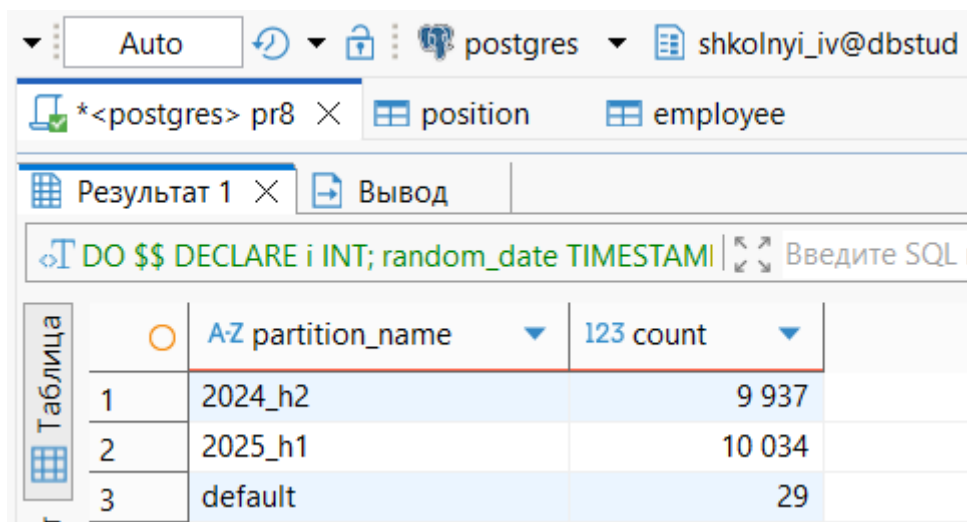


Таблица	partition_name	count
1	2024_h2	9 937
2	2025_h1	10 034
3	default	29

Рисунок 3 — Успешная генерация

Задание 4. Анализ и поиск по JSONB

Найти все бронирования, где сумма чека больше 80000 рублей.

Листинг 7. Запрос 1

```

SELECT
    log_id,
    created_at,

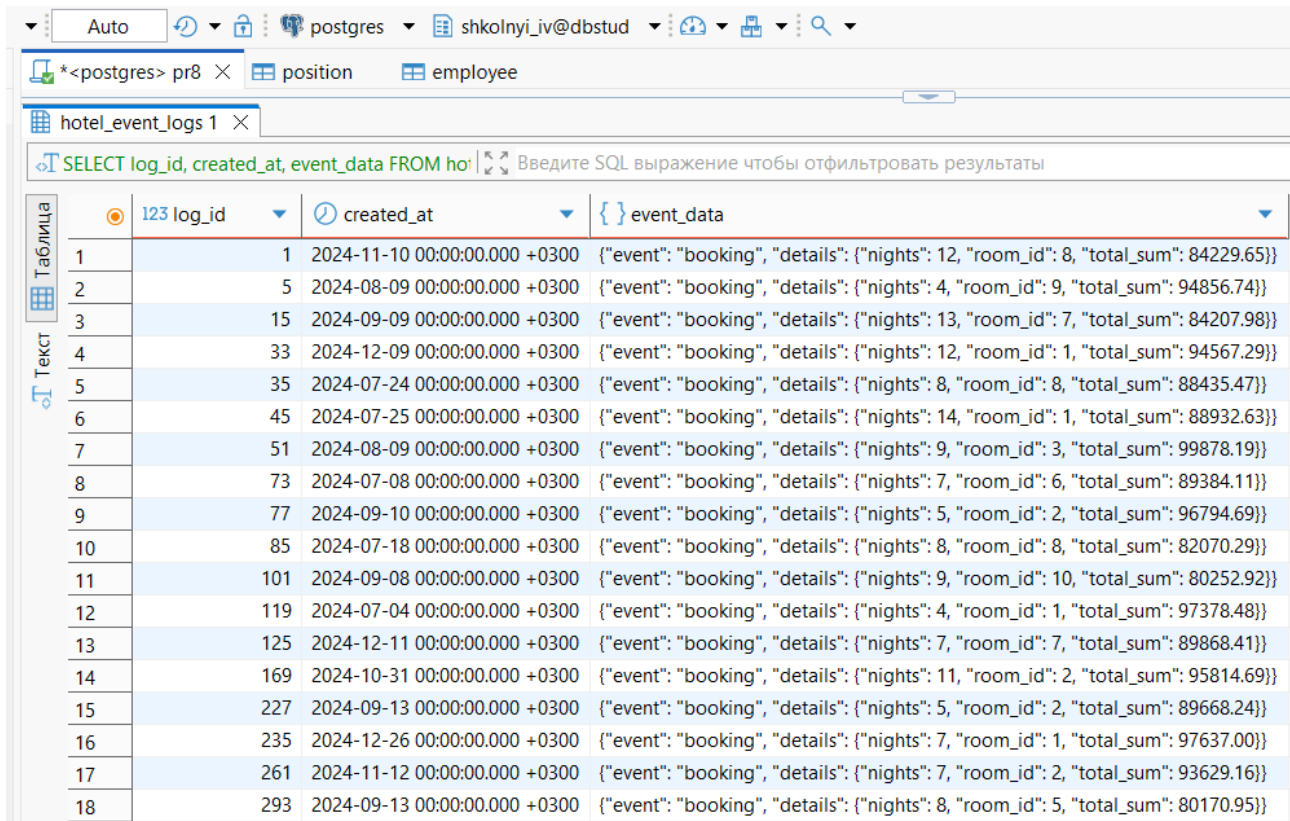
```

```

event_data
FROM hotel_event_logs
WHERE event_data -> 'event' = "'booking'"
AND (event_data -> 'details' -> 'total_sum')::NUMERIC > 80000;

```

Запустим скрипт.



	log_id	created_at	event_data
1	1	2024-11-10 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 12, "room_id": 8, "total_sum": 84229.65}}
2	5	2024-08-09 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 4, "room_id": 9, "total_sum": 94856.74}}
3	15	2024-09-09 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 13, "room_id": 7, "total_sum": 84207.98}}
4	33	2024-12-09 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 12, "room_id": 1, "total_sum": 94567.29}}
5	35	2024-07-24 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 8, "room_id": 8, "total_sum": 88435.47}}
6	45	2024-07-25 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 14, "room_id": 1, "total_sum": 88932.63}}
7	51	2024-08-09 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 9, "room_id": 3, "total_sum": 99878.19}}
8	73	2024-07-08 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 6, "total_sum": 89384.11}}
9	77	2024-09-10 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 5, "room_id": 2, "total_sum": 96794.69}}
10	85	2024-07-18 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 8, "room_id": 8, "total_sum": 82070.29}}
11	101	2024-09-08 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 9, "room_id": 10, "total_sum": 80252.92}}
12	119	2024-07-04 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 4, "room_id": 1, "total_sum": 97378.48}}
13	125	2024-12-11 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 7, "total_sum": 89868.41}}
14	169	2024-10-31 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 11, "room_id": 2, "total_sum": 95814.69}}
15	227	2024-09-13 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 5, "room_id": 2, "total_sum": 89668.24}}
16	235	2024-12-26 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 1, "total_sum": 97637.00}}
17	261	2024-11-12 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 2, "total_sum": 93629.16}}
18	293	2024-09-13 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 8, "room_id": 5, "total_sum": 80170.95}}

Рисунок 4 — Корректный 1 запрос

Найдем все записи, где количество ночей равно 7.

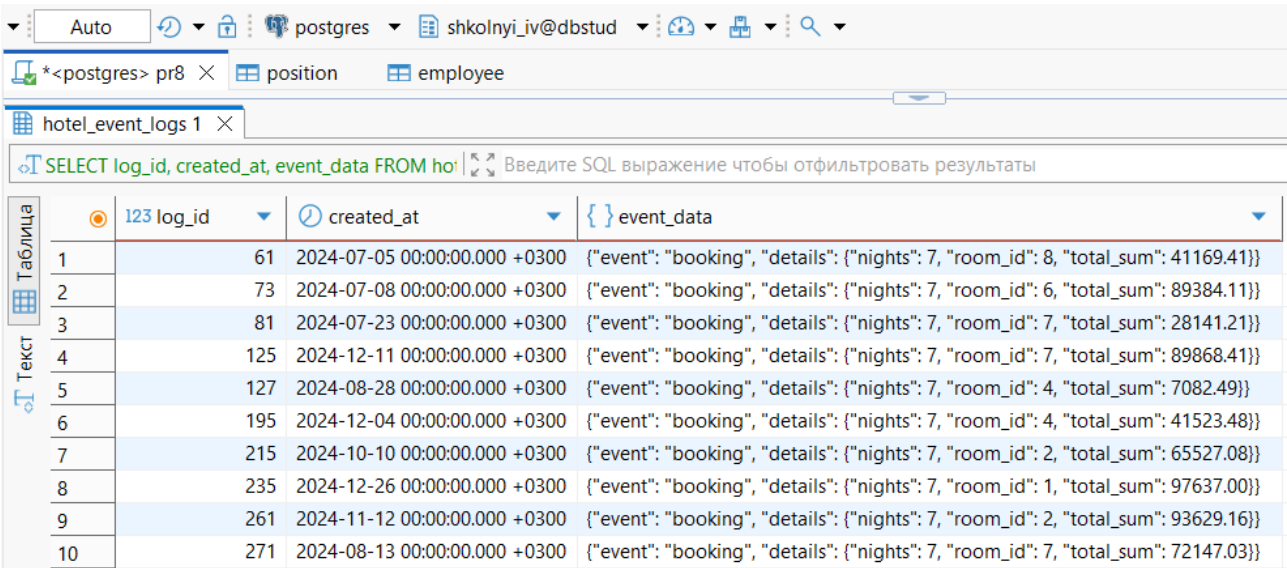
Листинг 8. Запрос 2

```

SELECT
  log_id,
  created_at,
  event_data
FROM hotel_event_logs
WHERE event_data @> '{"details": {"nights": 7}}'
LIMIT 10;

```

Запустим скрипт.



	log_id	created_at	event_data
1	61	2024-07-05 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 8, "total_sum": 41169.41}}
2	73	2024-07-08 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 6, "total_sum": 89384.11}}
3	81	2024-07-23 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 7, "total_sum": 28141.21}}
4	125	2024-12-11 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 7, "total_sum": 89868.41}}
5	127	2024-08-28 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 4, "total_sum": 7082.49}}
6	195	2024-12-04 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 4, "total_sum": 41523.48}}
7	215	2024-10-10 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 2, "total_sum": 65527.08}}
8	235	2024-12-26 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 1, "total_sum": 97637.00}}
9	261	2024-11-12 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 2, "total_sum": 93629.16}}
10	271	2024-08-13 00:00:00.000 +0300	{"event": "booking", "details": {"nights": 7, "room_id": 7, "total_sum": 72147.03}}

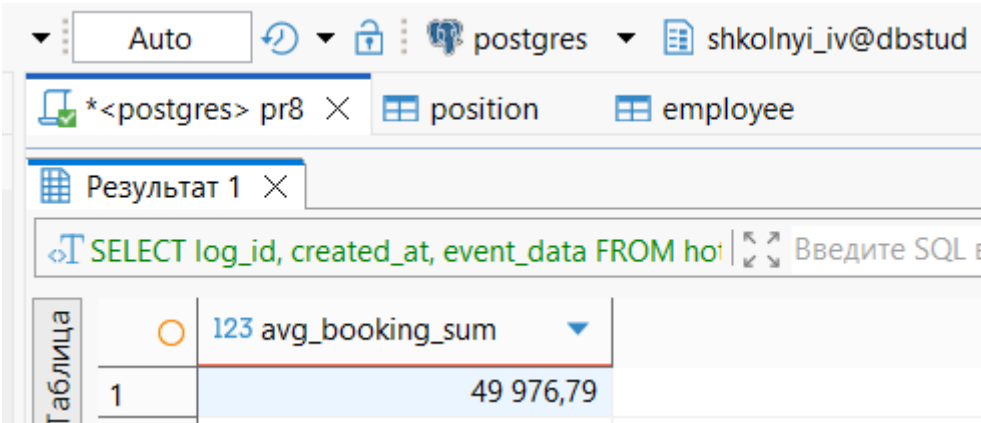
Рисунок 5 — корректный 2 запрос

Посчитаем среднюю сумму бронирования.

Листинг 9. Запрос 3

```
SELECT
  AVG((event_data -> 'details' -> 'total_sum')::NUMERIC)::NUMERIC(10,2) AS
  avg_booking_sum
FROM hotel_event_logs
WHERE event_data -> 'event' = 'booking';
```

Запустим скрипт.



avg_booking_sum
49 976,79

Рисунок 6 — Корректный 3 запрос

Задание 5. Модификация данных JSONB

Добавим поле "source": "system" во все логи. Оператор || объединяет два JSON-объекта. Если ключ уже существовал, он перезапишется. Если нет — добавится.

Листинг 10. Добавление поля

```
SELECT
```



```

    AVG((event_data -> 'details' -> 'total_sum')::NUMERIC)::NUMERIC(10,2) AS
    avg_booking_sum
FROM hotel_event_logs
WHERE event_data -> 'event' = "'booking'";

```

Запустим скрипт.

The screenshot shows a PostgreSQL client window with a table named 'hotel_event_logs'. The table has a column 'event_data' which contains JSON objects. The first row shows a booking event with details like nights, room_id, and total_sum. The second row shows a login event with ip, event, source, and success status. The third row shows another booking event.

id	event_data
1	{"event": "booking", "source": "system", "details": {"nights": 12, "room_id": 8, "total_sum": 84229.65}}
2	{"ip": "192.168.1.121", "event": "login", "source": "system", "success": true}
3	{"event": "booking", "source": "system", "details": {"nights": 4, "room_id": 9, "total_sum": 94856.74}}

Рисунок 7 — Добавленное поле source

Изменим статус успеха для одного входа (login).

Листинг 11. Изменение статуса

```

-- Найдём первый login и изменим success на false
UPDATE hotel_event_logs
SET event_data = jsonb_set(event_data, '{success}', 'false')
WHERE log_id = (
    SELECT MIN(log_id)
    FROM hotel_event_logs
    WHERE event_data ->> 'event' = 'login'
);

-- Проверяем
SELECT * FROM hotel_event_logs
WHERE event_data ->> 'success' = 'false'
LIMIT 3;

```

Запустим скрипт.

The screenshot shows the same PostgreSQL client window, but now the 'event_data' column contains JSON objects where the 'success' status has been updated to 'false' for the first login event. The table now has columns 'log_id', 'created_at', 'user_id', and 'event_data'.

id	log_id	created_at	user_id	event_data
1	2	2024-10-18 00:00:00.000 +0300	3	{"ip": "192.168.1.121", "event": "login", "source": "system", "success": false}
2	54	2024-10-06 00:00:00.000 +0300	7	{"ip": "192.168.1.115", "event": "login", "source": "system", "success": false}
3	72	2024-07-05 00:00:00.000 +0300	3	{"ip": "192.168.1.115", "event": "login", "source": "system", "success": false}

Рисунок 8 — Успешная работа

ВЫВОД

В ходе выполнения практической работы были освоены методы работы со сложными и неструктурированными типами данных (JSONB, BYTEA) в PostgreSQL, а также получены практические навыки реализации горизонтального масштабирования базы данных с помощью декларативного секционирования